3D Room Customizer

A guided tour of the room builder, furniture system, and editor

A practical explanation of how the current Floorplot code turns room dimensions and 3D models into an interactive room planner.

> **Source snapshot:** workspace code reviewed on 13 November 2026.
> **Coverage:** room presets and drawing, Three.js scene setup, model loading, placement, collision, interaction, UI, and undo.

How to use this guide

Read the app flow first, then follow a feature from its page input through the modules that handle it. Source file names are shown throughout so you can keep the code open in VS Code. Function explanations describe the current implementation rather than an idealized redesign.

This is a teaching guide, not a verbatim source-code listing. It explains every application function in the room/furniture/editor path and calls out the statements and algorithms that matter most. HTML layout, CSS decoration, tracking snippets, and static marketing content are not expanded line by line.

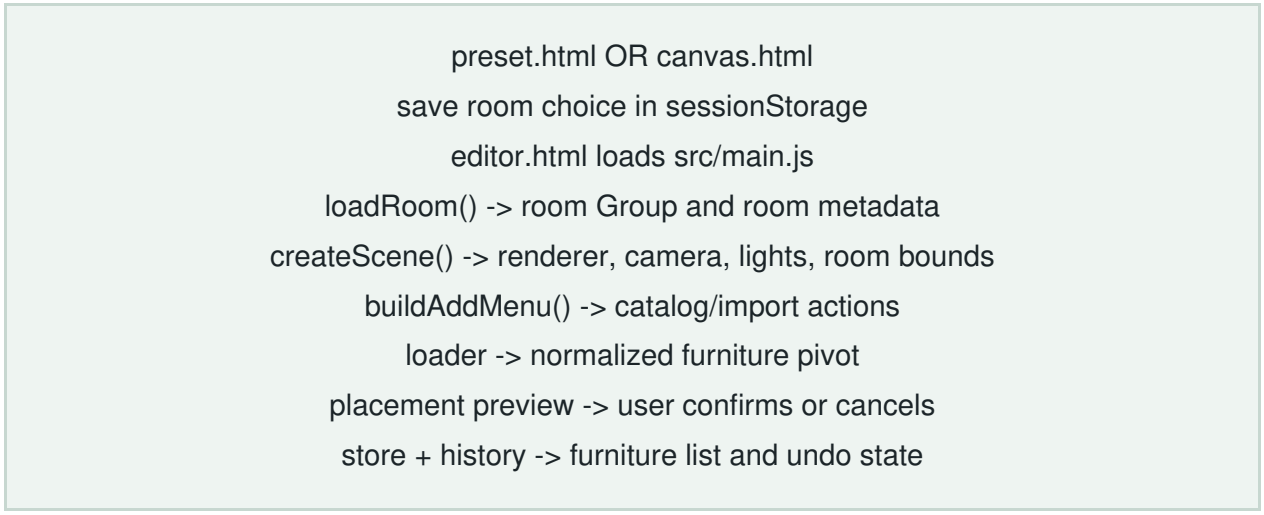
Contents

1. What the application does and how modules connect
2. Coordinate system, units, and shared data
3. Choosing and drawing a room
4. Loading and constructing room geometry
5. Creating the Three.js scene
6. Catalog furniture, load GLB models, and normalize size
7. Finding a first placement and tracking furniture
8. Geometry predicates and collision handling
9. Preview placement, dragging, and hover
10. Context menu, resize, and remove
11. Application wiring and camera controls
12. Undo, Add menu, and messages
13. Key JavaScript and Three.js ideas
14. Common changes, debugging, and limitations

1. What the application does

Floorplot is a browser-based room planner. The visitor chooses a preset room or draws an outline, the editor builds a Three.js scene, and furniture models can be loaded, placed, moved, rotated, resized, or removed. Most modules are small and focused; `src/main.js` wires them together.

End-to-end flow



The public landing page, `index.html`, links into the preset and drawing pages. `editor.html` supplies the scene container and controls; it loads the module graph from `src/main.js`.

Important layers

Layer	Responsibility	Files
Pages	Collect room input and provide editor controls	preset.html, canvas.html, editor.html
Room	Convert input into meshes and collision metadata	src/room/*, src/custom_model.js
Scene	Renderer, camera, lighting, controls, bounds	src/core/scene.js
Furniture	Catalog, GLB loading, sizing, collection, initial placement	src/furniture/*
Geometry	Floor plan tests and obstacle checks	src/physics/*
Interaction	Preview, drag, hover, context actions	src/interaction/*
Composition	Connect all services and run the render loop	src/main.js

2. Coordinate system and shared data

Units

Three.js coordinates are measured in metres by convention in this app. A room point is stored on the floor as `{ x, z }`; height is the Y axis. Furniture is moved along X/Z and normally has its pivot at Y = 0. The drawing canvas uses pixel-like `{ x, y }` points and converts them at the room-building boundary.

- `src/custom_model.js` uses `SCALE = 0.02` metres per canvas unit.
- `src/canvas.js` snaps to a 25-pixel grid and labels 25 pixels as 0.5 m.
- `src/room/presetRoom.js` uses 50 canvas units per metre, then the room builder converts those units with SCALE.
- `src/room/defaultRoom.js` is authored directly in scene units: 10 m wide, 10 m deep.
- `src/config.js` sets initial placement search steps to 0.25 m and collision tolerance to 0.0001.

Room object contract

A room builder returns a Three.js `Group`. Its meshes are rendered normally, while `group.userData` carries information other modules need:

- `outline`: ordered floor boundary points, each `{ x, z }`.
- `pillars`: polygons inside the outline, also in floor X/Z coordinates.
- `wallThickness`: used to leave a safe margin inside walls.
- `walls`: mesh references plus outward-facing normals, for camera-dependent visibility.

`main.js` wraps this metadata in a `collisionRoom` object and adds the room bounds and center calculated by `createScene()`.

Furniture object contract

A furniture item is a Three.js `Group` pivot containing the cloned GLB model. The pivot's position and rotation are used for placement. `userData.isFurniture` lets picking find the pivot; `userData.entry` retains its catalog/import metadata; `userData.fp` caches a floor footprint.

The same item object is held by the scene graph and by the furniture store. The scene graph renders it; the store is the list used for picking and collision tests.

3. Choosing and drawing a room

Preset page: `preset.html`

The page offers rectangle, square, and round/cylinder choices. `currentShape()` returns the selected radio value. `updateDimensions()` enables and requires only the inputs for that shape, hides the others, and redraws the preview.

`fmt(n)` prints whole numbers without a decimal and other values to one decimal. `label(x, y, text, anchor)` returns a small SVG text label. `updatePreview()` reads the selected dimensions, clears the preview if they are not positive, then generates a proportional rectangle or circle, labels its dimensions, and calculates area (`w * d` or `PI * r * r`).

On submit, the handler prevents normal form navigation, turns the form fields into an object, serializes it into `sessionStorage.presetRoom`, then navigates to the editor. This is a page-to-page handoff; the room meshes are built later.

Drawing page: `src/canvas.js`

The page first asks how many pillars the user will draw. Its submit handler stores that count, removes the setup overlay, then redraws the canvas. The drawing state is held in:

- `points` and `isClosed`: the room outline and whether its first point has been clicked again.
- `pillars`: completed pillar polygons.
- `currentPillar`: the pillar currently being drawn.
- `mousePoint`: snapped cursor position for preview feedback.

`snap(v)` rounds a coordinate to the nearest 25 pixels. `pixelsToMetres(pixels)` divides by 25, so one grid step represents half a metre. `resizeCanvas()` makes the backing canvas match window size at the device pixel ratio, then resets the drawing transform and redraws to keep lines crisp.

`drawMeasurement(start, end, color, preview)` computes Euclidean distance, converts it to metres, and draws a label; when preview is true it also draws a dashed segment. `drawShape(list, closed, color, fill)` connects points, optionally closes/fills the polygon, labels each edge, and draws corner

dots.

`nearFirstPoint(list, point)` returns true when a polygon has at least three points and the cursor is within 12 pixels of the first point. `redraw()` clears the canvas, redraws the room and pillars, displays the current dashed edge, highlights a possible closing point, sets the cursor, and updates the instruction panel. `updatePanel()` changes the stage title/hint and enables or disables Undo/Start over.

`undo()` removes the latest point; while drawing pillars it can reopen the last finished pillar or reopen the room outline. `startOver()` clears all drawing arrays and state. `finishRoom()` removes old preset data, stores outline and pillar points as JSON in `sessionStorage.customRoom`, and navigates to the editor.

Canvas click handling snaps the pointer; clicking near the first point closes the current outline/pillar. When the outline and required pillar count are complete, it calls `finishRoom()`. Otherwise it appends the point. Mouse move/leave update `mousePoint`; keyboard Backspace or Ctrl/Cmd+Z calls `undo()`.

4. Loading and constructing room geometry

src/room/loadRoom.js

`loadRoom()` checks saved input in priority order: preset, custom drawing, then the default room. Preset and custom JSON parsing/building are in try/catch blocks. A bad preset is logged and removed so it cannot repeatedly break the editor; bad custom data is logged and falls through to the default. A valid custom outline must have at least three points.

src/room/presetRoom.js

`rectanglePoints(width, depth)` creates four centered corners in drawing units using 50 pixels per metre. `cylinderPoints(radius)` samples 64 points around a circle, which approximates a smooth round room with a polygon.

`buildPresetRoom(preset)` converts form values to numbers, dispatches cylinder and square cases explicitly, and otherwise builds a rectangle when width and depth are positive. Invalid values throw an error; `loadRoom()` handles it.

src/room/defaultRoom.js

`buildDefaultRoom()` creates a group containing a floor and four box-shaped walls. Width/depth are 10, height is 4, and thickness is 0.2. The helper `addPart(w,h,d,x,y,z,material)` makes a box mesh, positions it, enables shadow reception, adds it to the group, and returns it. `addWall(..., normal, ...)` calls `addPart()` and records the mesh/normal pair.

The floor is centered slightly below Y=0, so its top is at ground level. Wall centers are at half their height. The group exposes a rectangular outline, empty pillars, wall thickness, and wall records in `userData`. This metadata is as important to placement as the visible boxes are to rendering.

src/custom_model.js

`buildCustomRoom(points, pillars)` is the shared constructor for drawn and preset rooms. It computes the drawing-space bounds center, then `toVector(point)` recenters and scales each point into metres. The signed polygon area determines winding so wall normals point outward for either point order.

A Three.js `Shape` becomes the floor; each pillar path is added as a hole. `ShapeGeometry` is rotated onto the X/Z ground plane and receives shadows. For every outline edge, the function calculates its length and angle, creates a thin wall box, positions it at half wall height, and records an outward normal.

Each pillar is turned into an extruded polygon with no bevel, rotated upright, and added as a mesh. At the end, the group stores walls, scaled pillar polygons, scaled outline, and wall thickness. Collision tests use those metadata polygons, not the decorative mesh geometry.

src/room/materials.js

`seeded(seed)` returns a deterministic pseudo-random number generator. Given seed 42, the same wood grain is generated on each run. `makeFloorTexture()` draws plank rows, seams, and grain strokes onto a 1024-pixel canvas; a `CanvasTexture` wraps/repeats in both directions and is marked sRGB. `FLOOR_TILE` determines the repeat scale in metres.

`makeFloorMaterial()` attaches that texture to a rough, nonmetallic, double-sided standard material. `makeWallMaterials()` returns six materials in BoxGeometry face order and darkens the top face, making cutaway walls read like an architectural model. `makePillarMaterial()` returns the brass-colored pillar material.

src/room/wallVisibility.js

`updateWallVisibility(room, camera)` updates world matrices, transforms each saved wall normal into world direction, and forms the vector from wall to camera. The dot product selects which side is visible. This makes the room easier to inspect from inside by hiding walls on the camera side.

5. Creating the Three.js scene

`src/core/scene.js` exports `createScene(container, room)`. It builds the WebGL renderer with antialiasing and transparency, sizes it to the container, caps pixel ratio at 2, configures tone mapping and soft shadows, then inserts the canvas into the page.

A `Scene` receives a PMREM environment generated from `RoomEnvironment`, which gives standard materials soft environmental reflections. The generator is disposed after producing the environment texture. A perspective camera and `OrbitControls` are created; damping smooths input, the polar-angle limit prevents orbiting under the floor, and distance limits control zoom.

Hemisphere and directional lights provide ambient and directional illumination. The directional light casts shadows and has a tuned shadow camera. Optional axes, grid, and light/camera helpers are added only when `SHOW_HELPERS` is true.

The room is added to the scene. Its outline is reduced to min/max X/Z bounds; the midpoint becomes the room center. Camera scale is based on the largest room dimension, and portrait layouts pull back farther so the room fits. Camera far plane, orbit distance, and light/shadow extents scale with the room. The function returns renderer, scene, camera, controls, light, bounds, and center coordinates.

6. Loading and normalizing furniture

src/furniture/catalog.js

`CATALOG` is plain data: unique ID, visible label, GLB path, and a target width or height. The Add menu and model loader both use these records. Built-ins include sofa, table, bed, lamp, desktop, and armchair.

src/furniture/loader.js

`loadModel(path)` caches the promise for each built-in path, not just the final model. If several requests arrive while a model is loading, they share one network load. A rejected load deletes the failed cache entry so a later try can succeed.

`buildFurniture(entry)` loads the catalog model and delegates to `buildFurniturePivot()`. `buildFurnitureFromFile(file, entry)` checks the .glb extension and nonzero file size, caches parsing by the browser File object in a `WeakMap`, and parses the ArrayBuffer with GLTFLoader. Failed parsing removes the cache entry.

`buildFurniturePivot(source, entry)` clones the source so each placed instance can be transformed independently. Every mesh is configured to cast and receive shadows. The optional initial Y rotation is

applied before measuring bounds. A `Box3` finds the model's dimensions. Imported geometry with empty or invalid X size is rejected.

The uniform scale is target height divided by model height when `entry.height` exists; otherwise it is target width divided by model width. After scaling, the model is centered in X/Z and its lowest point is moved to Y=0. It is placed under a new pivot group tagged with `isFurniture` and its metadata. Consistent pivots make picking, moving, rotating, and collision checks simpler.

Changing a built-in model

Add a catalog object with a unique ID, label, correct relative model path, and target width or height. Put the GLB in the model directory. The menu automatically chooses a built-in icon when its ID matches; otherwise it uses the generic icon. Verify the model bounds and orientation in the editor because different GLBs may use different origins/axes.

7. Tracking furniture and finding a first placement

src/furniture/store.js

`createFurnitureStore(onChange)` creates a private array and returns it as `items`, plus `add()` and `remove()`. `add(item, index)` ignores duplicates, clamps the insertion index, and calls the change callback. `remove(item)` removes an existing item, calls the callback if it changed the array, and returns its former index (or -1).

src/furniture/placement.js

`findFreeSpot(pivot, room, furniture)` computes other items' footprints once, then searches rotations 0 and 90 degrees. For each rotation it positions the candidate at the origin to get its relative footprint, scans the room bounds in `GRID_STEP` increments, and skips positions farther from the room center than the best valid candidate found so far.

Each candidate rectangle is tested against walls/outline/pillars using `fitsInRoom()`, and against existing pieces using `rectsOverlap()`. The returned record is `{ x, z, rot, distance }`. It returns null when no sampled position works. This is a grid search, so its speed/precision depend on grid spacing and room size.

8. Geometry predicates and collision handling

src/physics/geometry.js

These are small geometric helpers in floor coordinates, independent of Three.js and the DOM.

- `pointInPolygon(x,z,poly)` uses ray crossing: each edge crossing toggles an inside flag.
- `segmentHitsRect(a,b,r)` clips a line segment to rectangle boundaries. It returns true only for a nonzero segment through the rectangle's interior; a boundary touch alone is not treated as an interior crossing.
- `polygonHitsRect(poly,r)` checks whether any polygon edge crosses the rectangle, then checks whether the rectangle center lies inside the polygon. These cases cover overlap when either shape contains the other.
- `rectsOverlap(a,b,EPS)` compares X and Z intervals with a small tolerance so tiny numerical contacts do not count as overlap.

src/physics/collision.js

`footprint(obj)` uses `Box3.setFromObject()` to find the world-axis-aligned bounding box, then stores bounds relative to the pivot. It reuses that footprint while rotation is unchanged, and returns world position by adding the pivot location. Resizing explicitly clears this cache; movement does not need to because the relative size stays the same.

`fitsInRoom(rect, room)` expands the furniture rectangle by half the wall thickness. It rejects rectangles that cross the room boundary, whose center is outside the room polygon, or that overlap a pillar polygon. `isFree(item,room,furniture)` combines room fitting with overlap checks against every other item, excluding the item itself.

`nudgeToFree(item,radius,room,furniture)` samples offsets at 0.1 m increments around a rotated item and keeps the nearest free point. It restores the original position and returns false if no candidate works.

`tryOffset(item,ox,oz,...)` tries a full motion offset, then halves it up to six times to find a valid partial step; on total failure it restores the start. `moveWithCollision(item,targetX,targetZ,...)` splits long movement into steps no larger than about 0.2 m and tries diagonal movement, then X-only, then Z-only. This creates a sliding effect along obstacles.

> **Collision model:** furniture is represented by axis-aligned bounding rectangles on the floor, not exact mesh geometry. This is fast and predictable for room planning, but a long diagonal model can reserve more floor area than its visible shape. Vertical collisions are not modeled.

9. Preview placement, dragging, and hover

`src/interaction/placementPreview.js`

`installPlacementPreview(options)` creates a semi-transparent floor marker, a raycaster, and a horizontal floor plane. It keeps the currently pending furniture item and a saved camera pose.

`pointFromEvent(event)` maps browser pointer coordinates into normalized device coordinates, sets the ray from camera, and intersects it with the Y=0 floor plane. `update(event)` moves the pending pivot to that hit, snaps it near eligible walls, checks `isFree()`, and colors the footprint green or red.

`snapNearWalls(pivot)` calculates room winding and each edge's tangent/inward directions. It looks for a small inward offset that aligns the furniture bounding rectangle to the wall's inner face; a second pass lets it align against a corner's adjoining wall. The candidate must remain collision-free.

`begin(pivot,entry)` refuses a second simultaneous preview, calls `findFreeSpot()`, initializes rotation/position and marker bounds, saves the camera, disables orbit controls, and shows the placement hint. If no free grid spot exists it reports that and returns false.

`finish(confirmed)` leaves an invalid confirmed placement active and shows an error. Otherwise it hides the marker, re-enables camera controls, restores the saved camera, and either adds the pivot to the scene/store/history or simply discards the pending item.

> **Current controls:** move the outline with the pointer; left-click places it; right-click cancels it. Escape and window blur also cancel. This is implemented by a capture-phase pointerdown listener for the left button and a contextmenu listener for right-click.

`src/interaction/drag.js`

`installDrag()` owns the raycaster and active drag state. `setPointer(event)` converts screen pixels to normalized device coordinates. `pick()` intersects furniture meshes and walks up parent links until it finds the tagged furniture pivot.

`startDrag(item,event)` intersects the floor, records the starting position, disables orbit controls, computes the pointer-to-pivot offset, and shows a hint. `commitMovement()` records a history entry only if X/Z changed. `stopDrag()` commits, clears active state, restores orbit controls, and resets the cursor.

During pointer movement, the ray hits the floor and `moveWithCollision()` moves the item to the pointer while maintaining its initial drag offset. The R key commits the current movement, rotates 90 degrees, and calls `nudgeToFree()`; if no safe nearby position exists, the previous rotation/position are restored. Other keys stop the drag.

src/interaction/hover.js

`installHover()` creates a brass floor tint. Its internal `show(item)` sizes and centers that tint around the item's footprint, with 0.2 m extra width for visibility. Pointer movement is queued once per animation frame to avoid redundant raycasts; it hides the marker during touch, drag, or placement preview. Left-button/blur tracking prevents stale hover highlights while dragging.

10. Context menu, resize, and remove

`src/interaction/contextMenu.js` installs the furniture menu. Internal `close()` hides it and clears the target. `open(x,y,item)` records the target, places the menu inside the viewport, and focuses the first button. `addItem(label,hint,onSelect,className)` builds an accessible menu button; its click handler captures the target, closes the menu, then calls the action.

`rotate(item)` saves the current pose, rotates by 90 degrees, and tries to nudge to a free location. It restores the old pose and reports failure if blocked; otherwise it records a rotation behavior.

The menu actions are Move (starts drag), Rotate 90 degrees, Resize, and Remove. Remove drops the item from the scene and store, records its original array index for undo, and shows a toast.

`openDimensionDialog(item,left,top)` measures the current world bounding box, builds a form with width/height/depth inputs, and positions it near the menu. Inputs are text fields with decimal keyboard hints rather than number fields; this avoids browser step-mismatch popups for valid decimal values. `parseDimension(rawValue)` trims whitespace, accepts comma decimal separators by replacing commas with periods, and converts the result to a number.

Submitting prevents browser navigation, parses all dimensions, and rejects nonfinite or nonpositive values with an inline error. It scales each axis by requested/current world dimension, records the old and new scales for undo, invalidates the cached footprint, then closes the dialog. Escape or Cancel closes without changing the model. Pointerdown inside the dialog is stopped so it does not trigger canvas interactions.

`openAt(event)` ignores active drags, updates the picking ray, and opens the menu for the selected furniture. Right-click uses the browser contextmenu event; double-click offers the same menu on touchpads/touch devices. Clicking outside or Escape closes the menu.

11. Application wiring and camera controls

src/main.js

This is the composition root. It finds `#scene-container`, loads the room, creates the scene, and creates `collisionRoom` from room metadata plus calculated bounds and center. It calculates floor area with the polygon shoelace formula and subtracts pillar areas.

`updateMeta(items)` updates the editor header with floor area and piece count. `createFurnitureStore(updateMeta)` supplies the shared furniture array. `createBehaviorHistory()` updates the Undo button's disabled state and title whenever history changes.

`addFurniture(entry)` chooses catalog loading or file loading, then starts the preview. Errors are logged and surfaced with a message. `importFurniture(file,addEntry)` validates extension and nonzero size, creates metadata with a default target width of 2 m, loads the GLB, adds it to the dock, and starts preview.

The module then installs the add menu, drag, hover, placement preview, and context menu. All receive the same room and furniture references so collision and UI state stay consistent.

The initial camera position/target are cloned for Reset view. `setCameraView(view)` computes a target and positions the camera for room, top, or eye-level viewing. The view menu handlers update visibility/ARIA state, close when clicking outside, and close/restore focus on Escape.

The hint is dismissed on first context menu/double click or after 20 seconds. `animate()` schedules the next frame, updates wall visibility, updates damped orbit controls, and renders. The resize handler updates camera aspect/projection and renderer dimensions.

12. Undo, Add menu, and messages

src/undo/behaviorCatalog.js

`BEHAVIOR_CATALOG` pairs each behavior type with a label, undo callback, and redo callback. Place undo removes; remove undo restores at the original index; move, rotate, and resize restore recorded from/to values. `removeItem()` and `restoreItem()` keep scene and store synchronized.

`createBehaviorHistory({scene,store,onChange})` creates the stack and action context. The context setters change position, pose, or scale; scale changes clear cached footprints. `record(type,data)` rejects unknown types, pushes the action, and notifies the UI. `undo()` pops and applies the matching undo callback, then notifies the UI about the next item. `canUndo()` and the `length` getter expose stack state.

> **Current behavior:** redo callbacks are defined in the catalog, but the history API currently exposes only record/undo/canUndo/length and the editor has no Redo button. Do not assume redo is available in the UI.

src/ui/addMenu.js

`svg(inner)` wraps icon paths in a shared SVG shell. `sizeLabel(entry)` returns a height label or a width label. `buildAddMenu(catalog,onAdd,onImport)` builds a collapsible dock, catalog list, and hidden .glb file input.

Its internal `addEntry(entry)` builds the item button, icon, name, and size. Names use `textContent`, which treats imported file names as text instead of HTML. Clicking disables the option while the asynchronous load runs; `finally` always restores it and its label. Import button opens the file picker; change reads the selected file, resets the input so the same file can be selected again, and calls `onImport`.

src/ui/toast.js

`showMessage(text)` creates the shared accessible status element on first use, updates its text, shows it, clears the previous timer, and hides it after 2.5 seconds.

13. Key JavaScript and Three.js ideas

- **ES modules:** `import/export` make dependencies explicit. Pages use an import map to resolve `three` and `three/addons/`.

- **Promise caching:** caching the loading promise prevents duplicate in-flight fetches; each furniture instance still gets a clone.
- **Scene graph:** a Group can be transformed as one object and contains child meshes. The furniture pivot is the transform handle.
- **Vectors:** `Vector3` stores 3D coordinates; `Vector2` is used for pointer coordinates and 2D shapes.
- **Raycasting:** converts a screen pointer into a ray through the camera, then intersects furniture or the floor plane.
- **Bounding boxes:** `Box3` measures all child geometry in world space; the app uses it for model sizing and approximate floor collision.
- **userData:** a Three.js extension point for app metadata attached to an Object3D.
- **Optional chaining:** `undoButton?.addEventListener(...)` runs only if the optional DOM element exists.
- **Nullish coalescing:** `pillars ?? []` defaults only for null/undefined, not for valid empty arrays.
- **Event capture:** placement listens in capture phase so clicks can be claimed before other canvas controls see them.
- **sessionStorage:** saves room input across page navigation in the same tab. Furniture arrangements are currently held in memory, not persisted.

14. Extending and debugging the code

Add another built-in furniture item

1. Put the GLB in `models/` and verify its local axes and origin.
2. Add an entry to `src/furniture/catalog.js` with a unique ID, label, path, and target width or height.
3. Open the editor and test loading, initial placement, wall/pillar fit, rotation, resize, and undo.

Change room construction

Keep visible meshes and collision metadata in sync. If the room builder adds or changes walls, update `userData.outline`, `pillars`, `wallThickness`, and `walls` as appropriate. For custom rooms, preserve point winding and the canvas-to-metre scale.

Debugging checklist

- Model missing: check browser network/console errors and verify the GLB path is relative to the page URL.
- Model enormous/tiny: inspect the GLB's bounding dimensions and catalog target width/height.
- Placement rejected: check room outline winding/data, pillar polygons, wall thickness, and footprint bounds.
- Wall visibility looks wrong: check saved wall normals and camera-side dot product.
- Pointer misses the floor: check container size, camera projection after resize, and floor-plane intersection.
- Undo leaves stale collision: when changing scale, clear `item.userData.fp`; the history setter already does this.

Known limitations to keep in mind

- Floor collision uses rectangular bounding boxes, so it is an approximation rather than mesh-accurate physics.

- Initial placement is a discrete grid search; smaller grid steps improve precision but increase work.
- Built-in and imported models use uniform initial scaling, but the Resize dialog changes X/Y/Z independently.
- Room data persists in sessionStorage, but furniture placement and resize do not survive a page reload.
- The source has undo metadata for redo, but no redo operation is currently exposed.

File map

Path	What to open it for
src/main.js	Startup, orchestration, camera presets, render loop
src/canvas.js	Drawn room state and input
src/custom_model.js	Floor, wall, pillar mesh construction
src/room/	Room selection, presets, materials, wall visibility
src/core/scene.js	Three.js renderer, camera, lighting, bounds
src/furniture/	Catalog, model loader, store, initial placement
src/physics/	Floor geometry and collision tests
src/interaction/	Place, drag, hover, context menu, resize
src/undo/behaviorCatalog.js	Undoable operations
src/ui/	Add dock and transient messages

End of guide. For the exact current implementation, use the matching source file in the workspace; this PDF explains behavior and design intent but does not replace the source.